Digital Applications & Data Management

WS25/26

Dr. Jens Kohl

Roadmap Vorlesung

1. Einführung und Übersicht
2. Grundlagen Data Science
3. Vorgehen Data Science Use Case
4. Case Study Data Science
5. Grundlagen unüberwachtes Lernen
6. Grundlagen überwachtes Lernen (tabellarische Daten)
7. Case Study überwachtes Lernen (tabellarische Daten)
8. Grundlagen überwachtes Lernen (Bilddaten)
9. Case Study überwachtes Lernen und Transfer Learning (Bilddaten)
10. Grundlagen Generative AI
11. Generative AI mit Texten und Prompt Engineering
12. Agentic AI
13. Ausblick: Machine Learning in der Cloud und Reinforcement Learning

Vorlesung 12:
Agentic AI

Agents

Motivation

Automatisierung Workflows

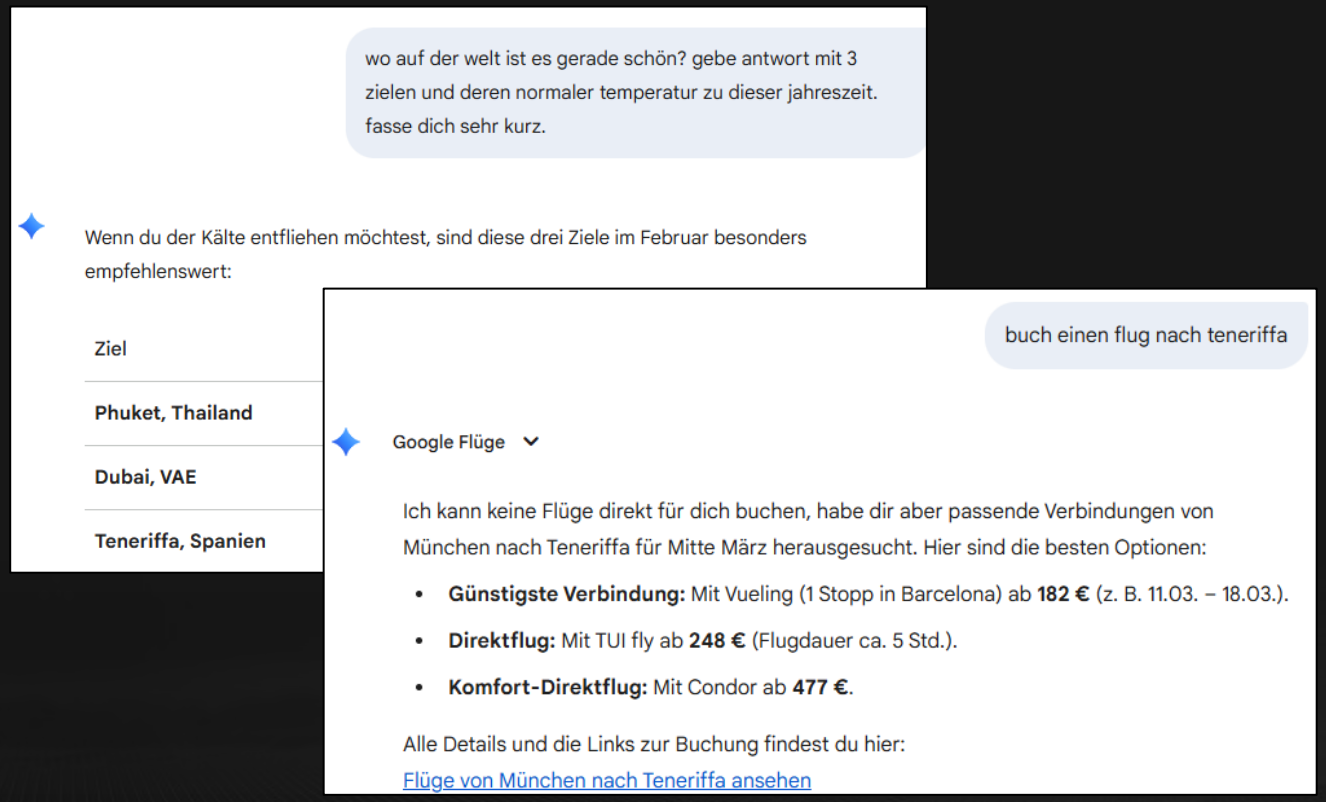
Source: https://aws.amazon.com/blogs/industries/bmw-group-develops-a-genai-assistant-to-accelerate-infrastructure-optimization-on-aws/, https://arxiv.org/abs/2506.11812

Coding assistants

Research assistants

Agentic AI

Ausgangssituation



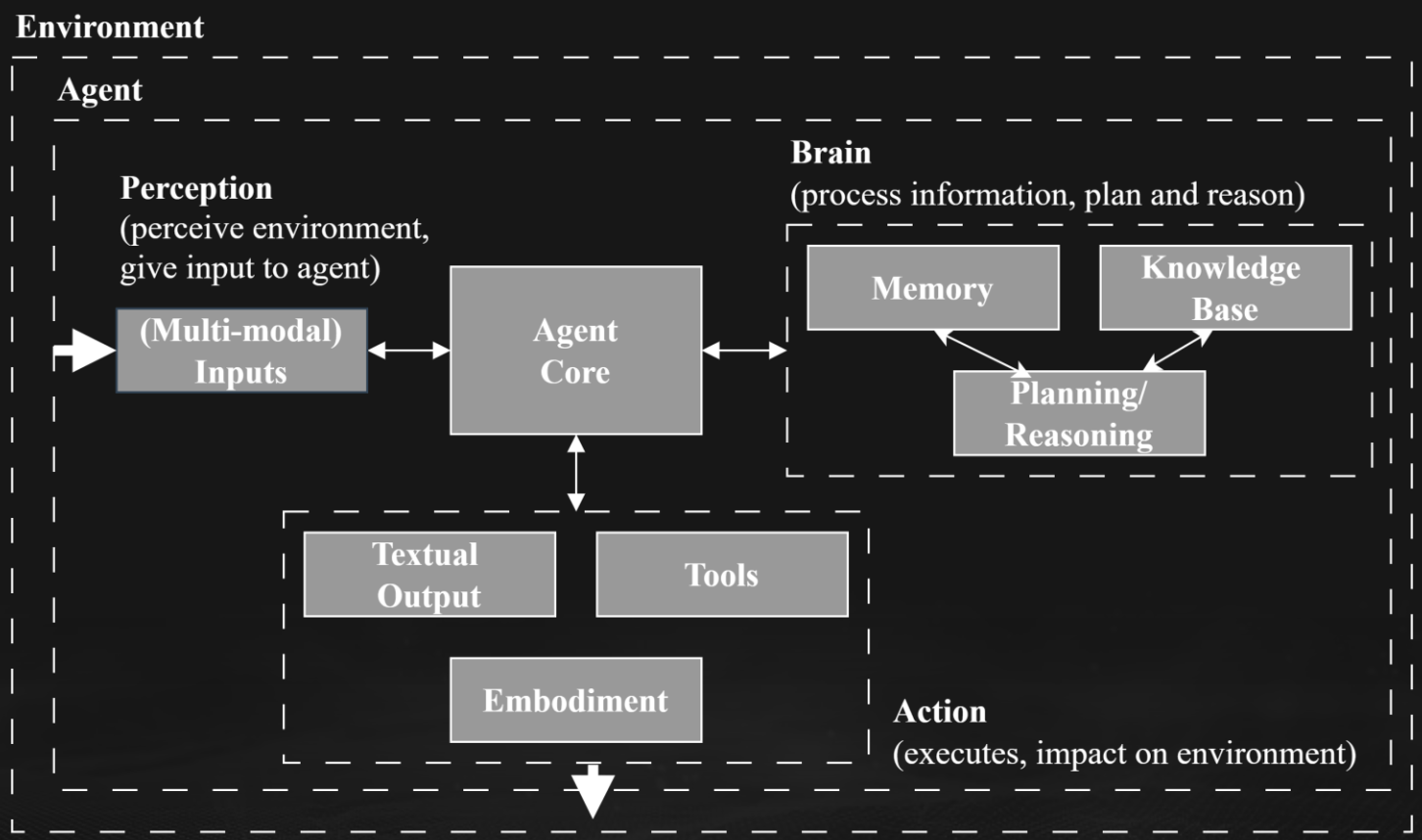
Grenzen klassischer LLM:

- Kein echtes Handeln/ Interaktion mit Umwelt/ Umgebung
- Kein dauerhaftes Gedächtnis
- Keine Feedback-Loop
- Kein Komplexen Plan

Cloud-basierte LLM Produkte werden aktuell so weiter entwickelt, daß sie diese Grenzen weitgehend überwinden können. Das Thema Interaktion mit Umwelt bleibt jedoch weiter offen….

Agentic AI

Struktur eines Agenten

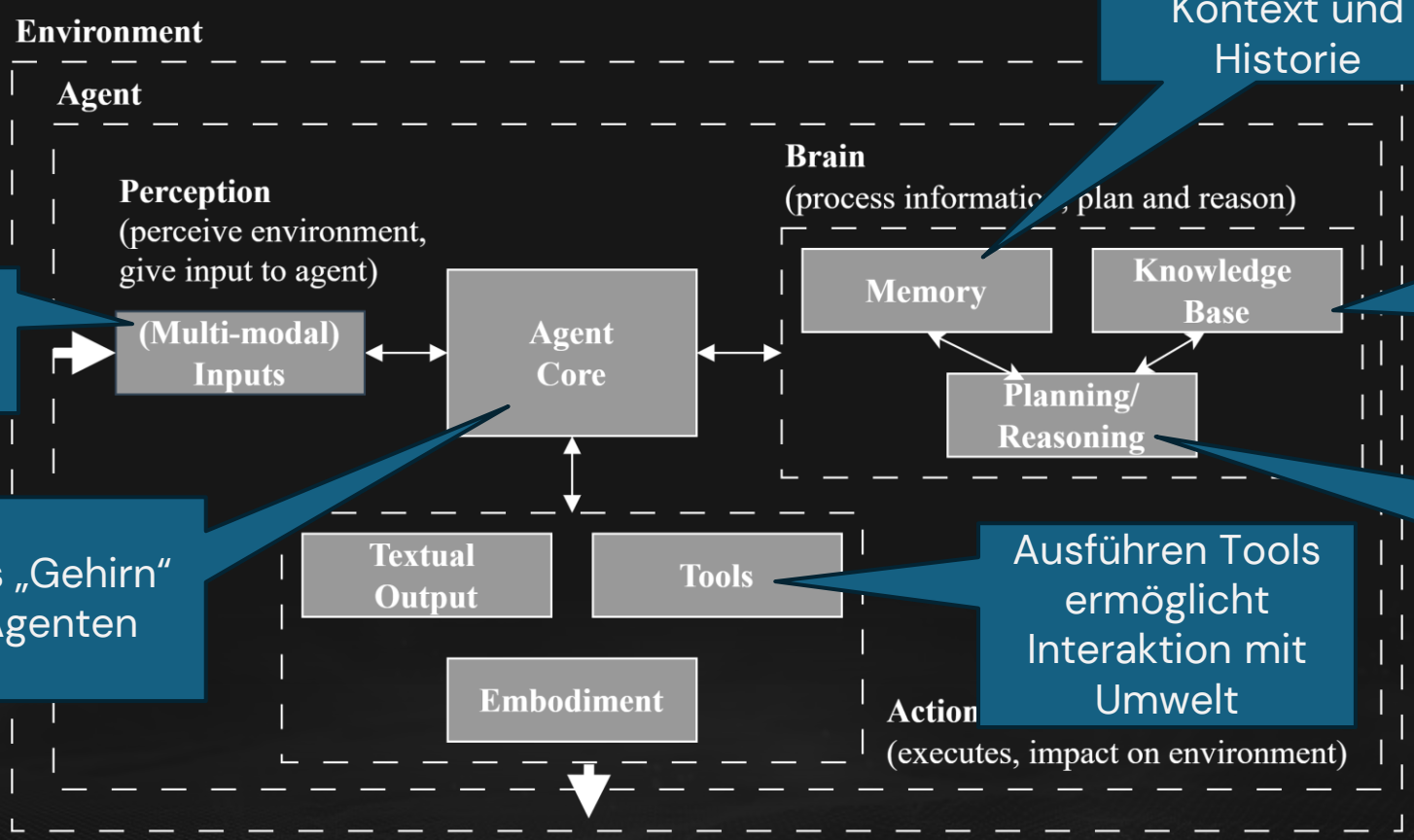


Agenten nutzen LLM als „Gehirn" und können durch Tools auch Aktionen ausführen

Quelle: Z. Xi et al., "The rise and potential of large language model based agents: a survey," Science China Information Sciences, vol. 68, no. 2, p.121101, 2025.

Agentic AI

Struktur eines Agenten



Speichern von Kontext und Historie

Benutzereingabe (Text, Bild)

Verfügbares Wissen (z.B. Dokumente, Datenbanken, ...)

LLM als „Gehirn“ des Agenten

Zerlegen von (komplexen) Aufgaben

Ausführen Tools ermöglicht Interaktion mit Umwelt

Agenten nutzen LLM als „Gehirn“ und können durch Tools auch Aktionen ausführen

Quelle: Z. Xi et al., “The rise and potential of large language model basedagents: a survey,” Science China Information Sciences, vol. 68, no. 2, p.121101, 2025.

Agentic AI

Detaillierung Inputs

- **Input-Quellen (Modalitäten):**
  - Text, Dokumente, Bilder, Video und Audio
  - Kontext-Variablen wie Zeit, Ort, User-Name.
- **Input Transformation:**
  - **Encoding:** Überführung Input-Daten in einen gemeinsamen Vektorraum (Embedding)
  - **Tokenisierung:** Zerlegung von Text in Token, d.h. numerische Vektoren
- **User Intent & Persona:**
  - **Task:** Definition konkrete Zielsetzung, was der Agent machen soll, in natürlicher Sprache.
  - **Rolle:** Definition des Verhaltensmusters (z. B. „Handle als Forscher“).
- **Constraints (Leitplanken):**
  - Explizite Verbote, einzuhaltende Regeln, Guardrails (Filter problematische Eingabetexte)
  - Zielvorgabe für den Output (z. B. JSON, Tabelle).
- Verknüpfung User-Inputs mit Instruktionen des Agenten (Task, Rolle) um Agentic-Loop zu starten

Agentic AI

Detaillierung Agentic Core

- LLM ist Gehirn des Agenten mit folgenden Aufgaben:
  - **Intent Recognition:** Erkennen, was der User wirklich will.
  - **Planung:** Zerlegen der User Anfrage in Teilaufgaben (chain-of-thought)
  - **Tool selection:** Identifizieren, welches Tool für welche Aufgabe geeignet ist
  - **Stop-Kriterium**: Agenten sind in einer Dauerschleife bis die Aufgabe erfüllt ist.
- **Anforderungen an das Modell:**
  - Großes Kontext-Fenster: Muss Historie, Tool-Beschreibungen, Dokumente, User inputs, … gleichzeitig verarbeiten.
  - Modell sollte Reasoning Fähigkeiten haben (bspw. Anthropic)
  - Striktes Einhalten von Ausgabeformaten für Interaktion mit Tools (wenn Tool nur ein spezifisches Eingabeformat hat, muß das genau so bedient werden)
- Temperatur wird oft auf 0 gesetzt, um „Kreativität“ zu unterbinden

Agentic AI

Detaillierung Planning & Reasoning

- Strategische Zerlegung (Decomposition):
  - Modell generiert Zwischenschritte vor Ausführen einer Aktion (Chain-of-Thought[1]).
  - Dies ermöglicht höhere Genauigkeit bei komplexen Aufgaben.
- ReAct-Framework (Reason + Act[2]):
  - Thought: Analyse aktuelle Situation
  - Action: Gezielter Aufruf eines Tools
  - Observation: Einbeziehung externes Feedbacks (z. B. Suchergebnis/ Fehlermeldung).
  - Somit ständiger Abgleich zwischen Planung und Realität/ Feedback möglich.
- Reflexion & Fehlerkorrektur (Self-Correction):
  - Agent bewertet Zwischenergebnisse auf Plausibilität.
  - Bei Tool-Fehlern oder Sackgassen wird Plan modifiziert, statt abzubrechen.
- Plan-and-Execute vs. Dynamisches Planning:
  - Plan-first: Erstellung eines kompletten Ablaufplans vor der ersten Aktion (effizient für Standard-Tasks).
  - Dynamisch: Schritt-für-Schritt Planung, bei der jeder nächste Schritt von der letzten Beobachtung abhängt.

Quelle:
1 Wei et al.: "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", 2022
2 Yao et al.: "ReAct: Synergizing Reasoning and Acting in Language Models", 2022.

Agentic AI

ReAct Loop am Beispiel Wetter Suche

```python
from langchain_openai import ChatOpenAI
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.messages import HumanMessage, ToolMessage

# 1. Setup Tool and LLM
search = DuckDuckGoSearchRun() # we use DuckDuckGo as search engine
tools = [search] # define tools, in our case just 1
llm = ChatOpenAI(model="gpt-4o").bind_tools(tools) # register tool

def run_manual_react(query: str):
    messages = [HumanMessage(content=query)] # Initialize message history

    while True:
        # --- STEP 1: REASONING (The "Thought") ---
        response = llm.invoke(messages)
        messages.append(response)
        # If the LLM doesn't want to use a tool, it's done reasoning
        if not response.tool_calls:
            break

        # --- STEP 2: ACTING (The "Action") ---
        for tool_call in response.tool_calls:
            print(f"Action: Searching for '{tool_call['args']['query']}'...")
            result = search.invoke(tool_call["args"]) # Execute the tool call

            # --- STEP 3: OBSERVATION (The "Observation") ---
            messages.append(ToolMessage(tool_call_id=tool_call["id"], content=result)) # Feed tool result back into agent conversation
            print(f"Observation: Found data. Summarizing...")
    return messages[-1].content

# we now want to call the agent for a specific city
city = "Munich"
answer = run_manual_react(f"What is the current weather and a top news story in {city}?")
print(f"\nFinal Answer:\n{answer}")
```

Agentic AI

Detaillierung Memory

Short-term Memory:

- **Inhalt:** Aktueller Dialogverlauf, Tool-Feedback und Denkprozesse.
- **In-Context Learning:** Modell „lernt“ aus Infos aus Prompt, ohne internen Gewichte zu verändern
- Memory wird durch Länge Context Window LLM begrenzt (z. B. 1M Token bei Claude)
- **Context Optimization (Management-Strategien)**
  - **Sliding Window:** Fokus auf N aktuellsten Token, Rest wird gelöscht
  - **Summarization:** bisherige Verlauf wird komprimiert zusammengefasst

Long-term Memory:

- Inhalt: Langfristiges Speichern von Wissen, Historie und Nutzerpräferenzen über verschiedene Sessions hinweg.
- Ablage: Daten werden langfristig gespeichert in
  - Klassische Datenbanken: für exaktes Suchen
  - Knowledge Graphen: Speichern von Beziehungen von Daten untereinander
  - Vektor-Datenbanken: ermöglichen semantische Suche (d.h. Ähnlichkeit)

Ziel ist das Sicherstellen der Kohärenz über lange, komplexe Workflows sowie Vermeiden redundanter Tool-Aufrufe und wiederholter Fehler.

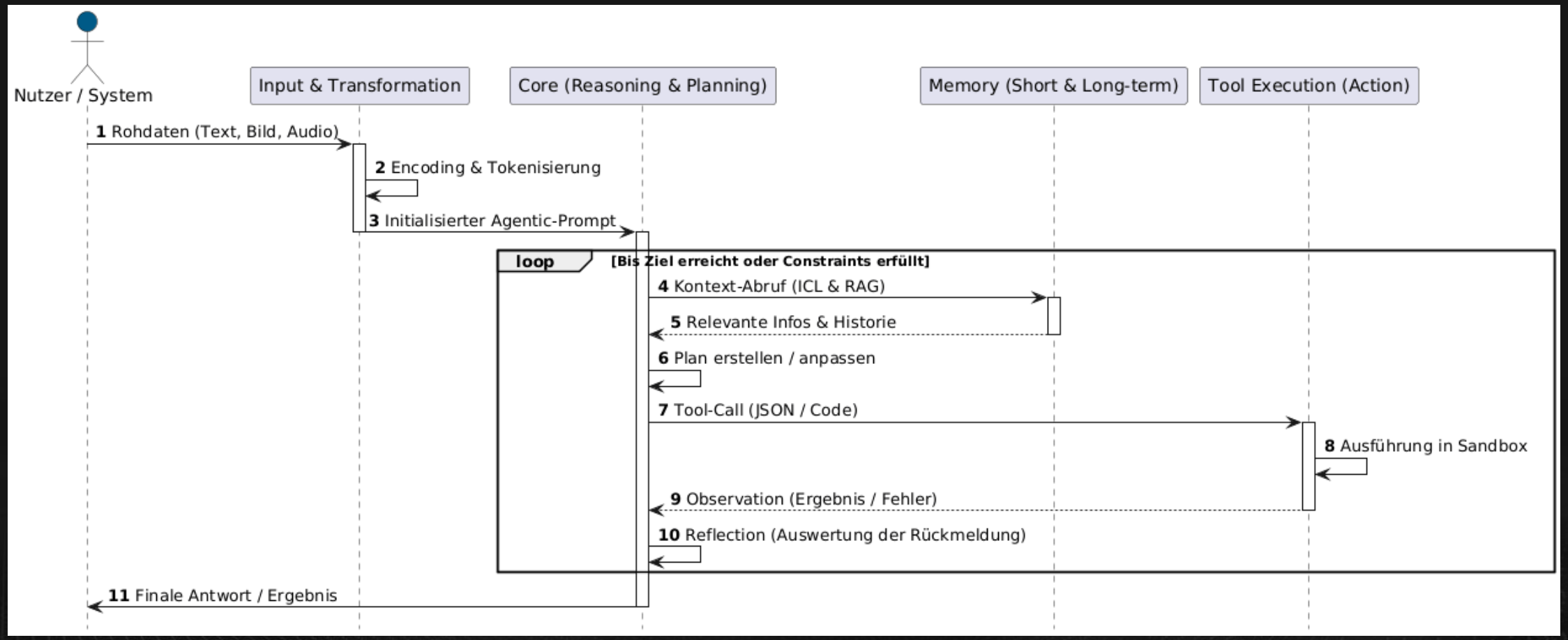
Agentic AI

Detaillierung Tools

- Registrierung Tools für den Agenten:
  - Tools werden LLM über präzise Meta-Daten (Name, Parameter, Beschreibung) „vorgestellt“.
  - Tools werden über standardisierte Schnittstellen oder über generierten Code aufgerufen.
- Auswahl Tool durch Agenten:
  - Modell wählt basierend auf dem aktuellen Plan das passende Werkzeug.
  - Tool erhält Input gemäß seines gewünschten Formats
- Ausführung Tool durch Agenten:
  - Ausführung von Code in isolierten Umgebungen zur Sicherheit (Sandbox)
  - Ergebnis Aktion (Erfolg/Fehler) wird erfaßt.
- Auswertung Rückmeldung Tools durch Agenten
  - „Observation“ wird als neuer Kontext dem Reasoning-Prozess des LLM zurückgeführt.
  - Bei Fehlern (z. B. Timeout) entscheidet Agent autonom über einen Retry oder Strategiewechsel.

Agentic AI

Wie alles zusammenspielt



Agentic AI

Herausforderungen

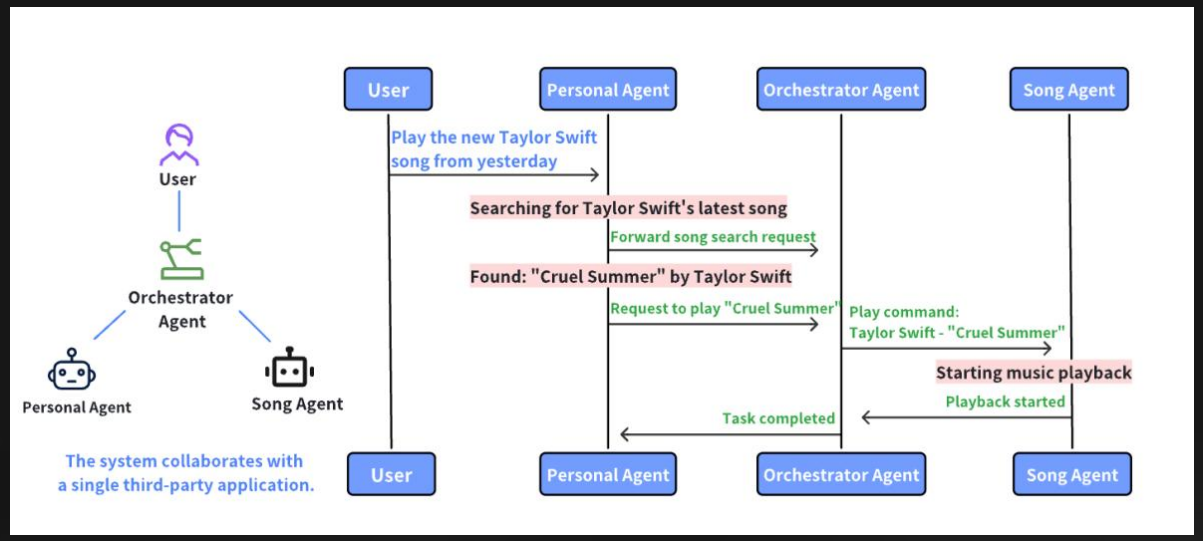
- **Zuverlässigkeit & Vorhersagbarkeit:**
  - Agenten können fiktive Tools erfinden oder falsche Rückmeldungen fehlinterpretieren (**Halluzinationen)**
  - Kleine Änderungen im Prompt oder Input können zu unterschiedlichen Handlungsplänen führen.
- **Effizienz & Kosten:**
  - Loop (Reasoning -> Action -> Observation) benötigt Zeit und ist langsamer als einzelnes LLM
  - Iterationen und das Anwachsen Historie führen zu hohen Nutzungskosten (Verrechnung auf Token-Basis!)
- Sicherheit (Security):
  - **Prompt Injection:** internen "Leitplanken" (Constraints) des Agenten umgehen.
  - Missbrauch bei Schreibzugriffen auf Datenbanken oder Dateisysteme
- **Komplexität der Evaluierung:**
  - Schwierigkeit, Agenten systematisch zu testen, da der Pfad zur Lösung jedes Mal variieren kann.
  - Risiko, dass der Agent in einer Endlosschleife aus Fehlern und Korrekturversuchen hängen bleibt.

Auf absehbare Zeit wird man aufgrund dieser Herausforderungen immer noch einen Menschen als letzte Instanz für kritische Umfänge benötigen („Human-in-the-Loop“)

Agentic AI

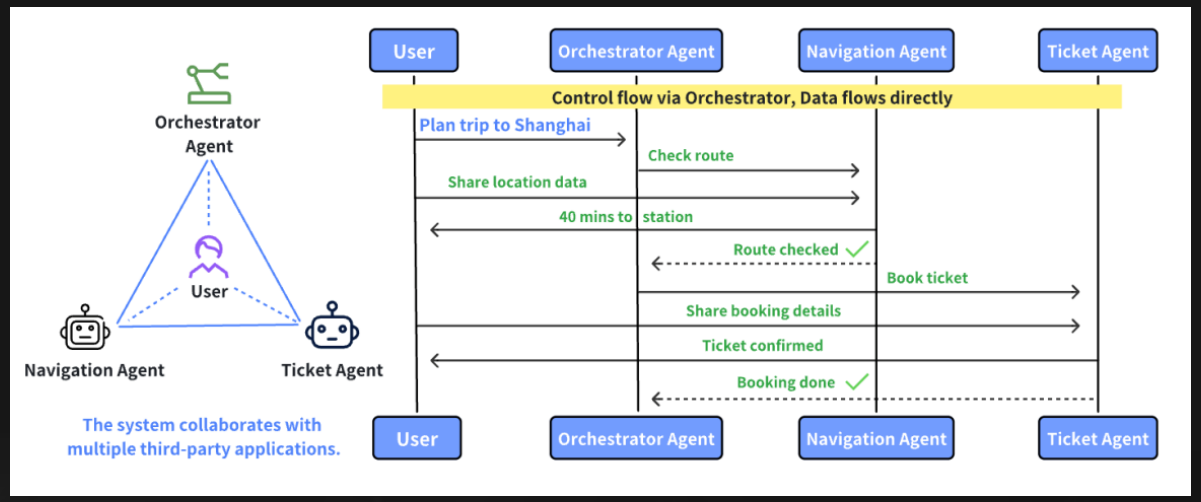
Multi-agenten System

Hierarchische Kollaboration:



1 zentraler Agent interagiert mit User, delegiert und führt Ergebnisse dann zusammen.

Sequentielle Kollaboration:



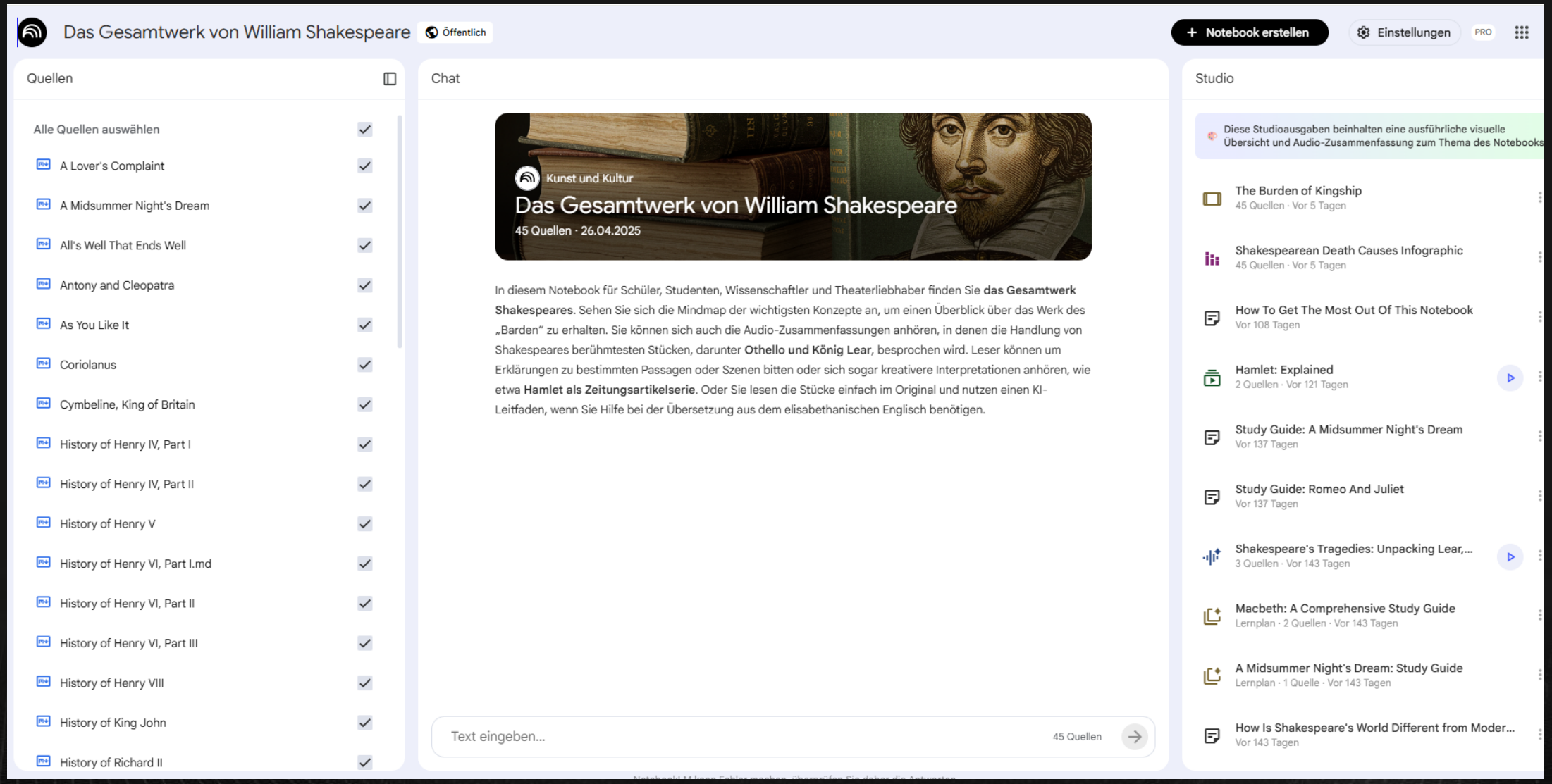
Agent A liefert Output, der für Agent B Input ist....

Falls Aufgaben für einen Agenten zu komplex oder fehleranfällig werden, kann man mehrere einzelne Agenten mit festen Rollen verwenden, die gemeinsam kollaborieren.

Quelle: Yang et al. (2024), “LLM-based Multi-Agent Systems: Techniques and Business Perspectives”

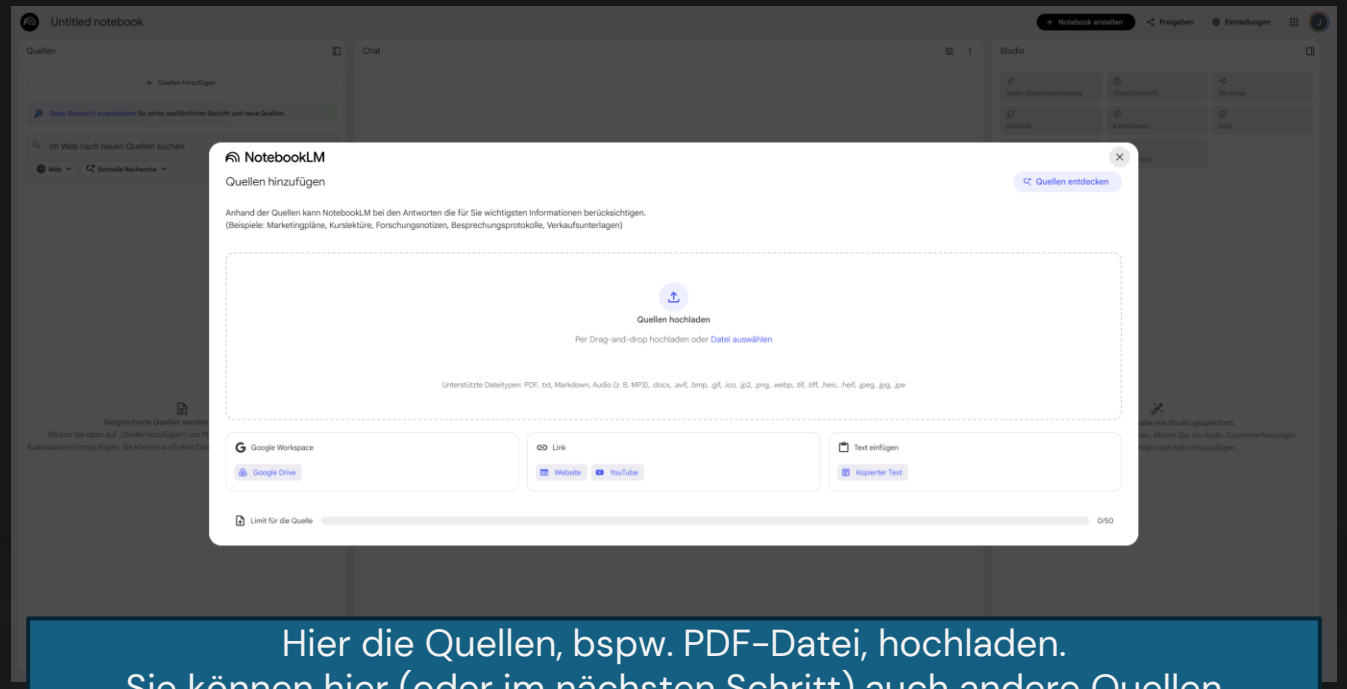
Case studies

Anwendungsbeispiel: Notebook LLM



Notebook-LLM: Use Case Analyse größerer Dokumente

Laden der Dokumente



Hier die Quellen, bspw. PDF-Datei, hochladen.
Sie können hier (oder im nächsten Schritt) auch andere Quellen hochladen oder Google suchen lassen

Quellen: https://x.com/thisdudelikesAI/status/2004842229140205595 , https://x.com/hasantoxr/status/2005553248971710625

Notebook-LLM: Use Case Analyse größerer Dokumente

Mögliche Prompts für effizientes Auswerten

1/ Get an overview
"Give me an overview of this chapter. What is the main idea? What are the key concepts?"
"Read this PDF like a teacher, not a summarizer. Explain the core idea first, then build up slowly. Assume I'm smart but new to this topic."
"I uploaded this PDF. Give me a high-level overview of the entire document, broken into key themes and concepts, as if you're introducing it to someone seeing it for the first time."
"Turn the main ideas of this document into structured outlines or mental models that help me see how everything connects."

2/ Turn Chapters Into Classes
"Turn this chapter into a 30-minute lesson plan. Include sections, examples, and quick checks for understanding."
"Turn each section or chapter of this document into a short lesson with a clear explanation, examples, and a quick summary at the end."
"Teach the content of this document step by step, starting from the basics and gradually increasing difficulty. Assume I'm learning this subject for the first time."

3/ Active Learning Mode
"Pause after every major concept and ask me a question. If I answer wrong, explain again using a different analogy."
"Create practice questions from this document that test my understanding. Ask me questions first, then explain the correct answers after."
"Extract all important concepts, terms, and definitions from this document and explain each one clearly, as if I need to remember them for an exam."

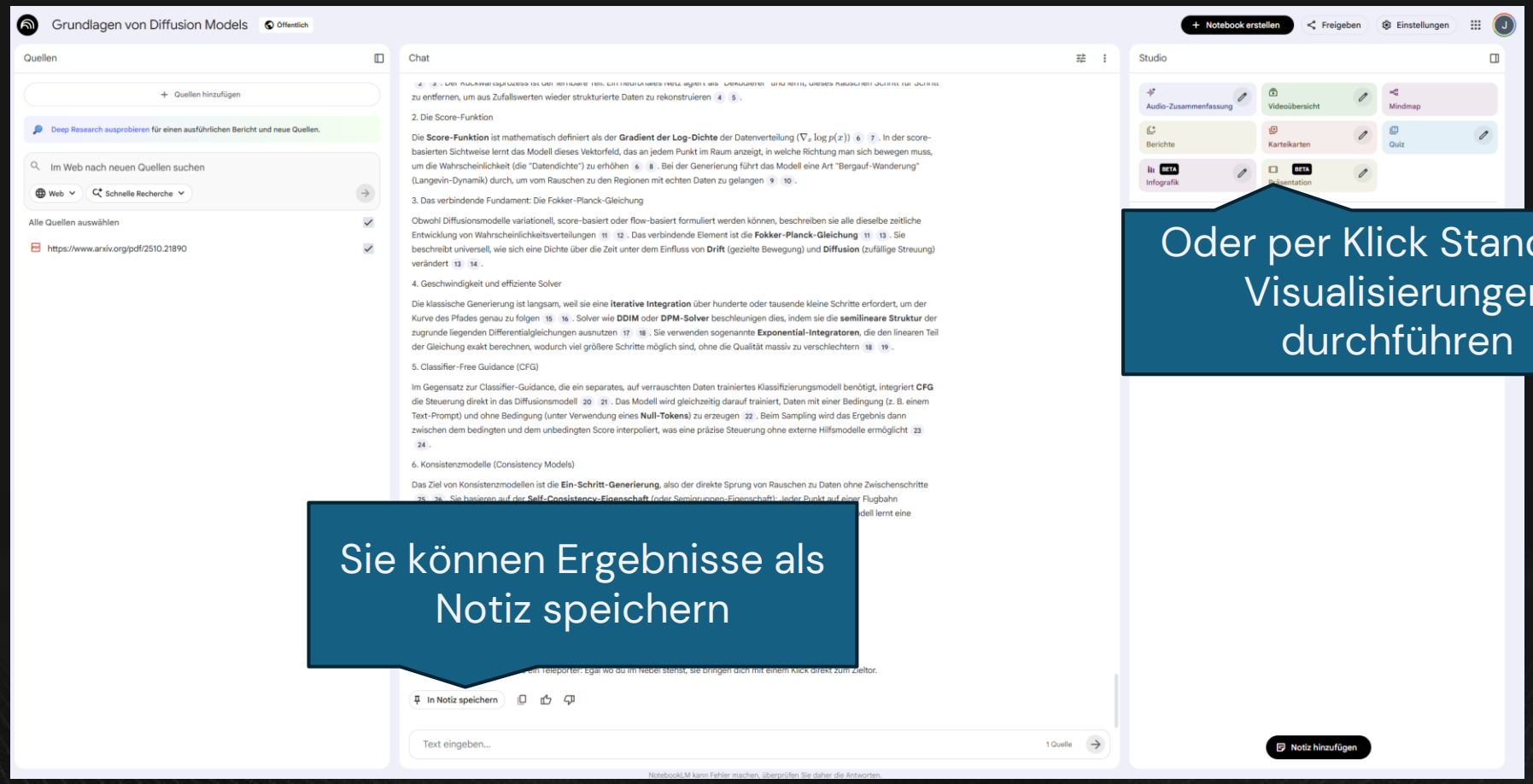
4/ Cut the Academic Noise
"Extract the 10 ideas from this PDF that actually matter in real life. Ignore filler, theory padding, and academic fluff."
"Identify the most complex or confusing parts of this document and explain them in simple language using analogies or real-world examples."
"Rewrite this document into concise study notes optimized for quick revision, highlighting only what actually matters."

5/ Teach for Application, Not Exams
"Teach this document using real-world examples from [your field]. Assume I want to apply it, not memorize it."

6/ Examiner Mode
"Act like an examiner and test me on this document. Then explain what my answers reveal about my understanding gaps."
"Pretend I just studied this document. Ask me questions and check my answers. If I'm wrong or unclear, correct me and explain what I missed."

Notebook-LLM: Use Case Analyse größerer Dokumente

Visualisieren Auswertungen



Quellen: https://x.com/thisdudelikesAI/status/2004842229140205595 , https://x.com/hasantoxr/status/2005553248971710625

Research multi-agent

Details

```python
root_agent = Agent(
    name="ResearchCoordinator",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        api_key=GEMINI_API_KEY,
        retry_options=retry_config
    ),
    # instruction tells root agent HOW to use its tools (which are the other agents).
    instruction="""You are a research coordinator. Your goal is to answer the user's query by orchestrating a workflow.
    1. First, you MUST call the `ResearchAgent` tool to find relevant information on the topic provided by the user.
    2. Next, after receiving the research findings, you MUST call the `SummarizerAgent` tool to create a concise summary.
    3. Finally, present the final summary clearly to the user as your response.""",
    # wrap sub-agents in `AgentTool` to make them callable tools for the root agent.
    tools=[AgentTool(research_agent), AgentTool(summarizer_agent)],
)
```

```python
research_agent = Agent(
    name="ResearchAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        api_key=GEMINI_API_KEY,
        retry_options=retry_config
    ),
    instruction="""You are a world class specialized research agent.
    Your only job is to use the Google_search tool to find 2-3 pieces of
    relevant information on the given topic and present the findings with
    citations.""",
    tools=[google_search],
    # Results of this agent will be stored in the session state with this name.
    output_key="research_findings",
)
```

```python
summarizer_agent = Agent(
    name="SummarizerAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        api_key=GEMINI_API_KEY,
        retry_options=retry_config
    ),
    # instruction modified to request a bulleted list for a clear output format.
    instruction="""Read the provided research findings: {research_findings}.
    Create a concise summary as a bulleted list with 3-5 key points.""",
    output_key="final_summary",
)
```

```python
print(response)
```

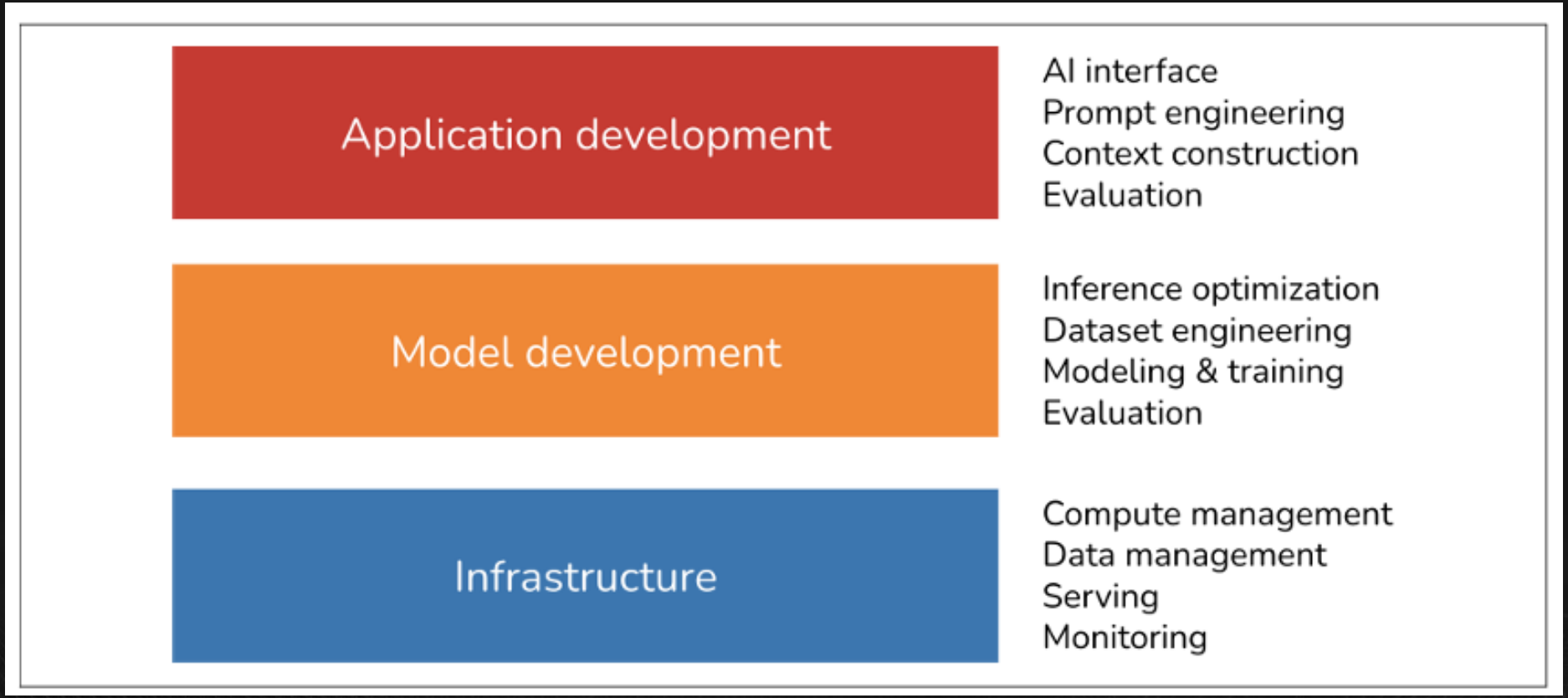
```
[Event(model_version='gemini-2.5-flash-lite', content=Content(
  parts=[
    Part(
      function_call=FunctionCall(
        args={
          'request': 'latest advancements in Diffusion models and their implications for AI'
        },
        id='adk-f4564cd5-53ec-42bc-bc54-bf1128a6f0a8',
        name='ResearchAgent'
      )
    ),
  ],
  role='model'
), grounding_metadata=None, partial=None, turn_complete=None, finish_reason=<FinishReason.STOP: 'STOP'>, error_code=None, error_message=None, interrupted=None, custom_metadata=None, usage_meta
  candidates_token_count=23,
  prompt_token_count=190,
  prompt_tokens_details=[
    ModalityTokenCount(
      modality=<MediaModality.TEXT: 'TEXT'>,
      token_count=190
    ),
  ],
  total_token_count=213
), live_session_resumption_update=None, input_transcription=None, output_transcription=None, avg_logprobs=None, logprobs_result=None, cache_metadata=None, citation_metadata=None, invocation_id
  parts=[
    Part(
      function_response=FunctionResponse(
        id='adk-f4564cd5-53ec-42bc-bc54-bf1128a6f0a8',
        name='ResearchAgent',
        response={
          'result': """The latest advancements in diffusion models are significantly enhancing their capabilities in generating high-quality and realistic content, including images, videos, an

Key advancements and their implications include:

*   **Improved Generative Quality and Control:** Diffusion models now achieve state-of-the-art performance in image and audio generation, often surpassing traditional models like Generative Ad
*   **Enhanced Efficiency and Accessibility:** Researchers are focusing on improving the computational efficiency of diffusion models to reduce processing time and energy consumption. Technqu
*   **Expanding Applications:** Beyond image and video generation, diffusion models are finding applications in diverse fields such as drug discovery (generating molecular structures), scienti
*   **New Model Architectures and Theoretical Insights:** Innovations like the Poisson Flow Generative Model++ (PFGM++), which integrates diffusion with Poisson Flow principles, are pushing th

The implications for AI are profound, as diffusion models are revolutionizing generative AI development and enabling more personalized and intelligent automation across various industries. The
        }
      )
    ),
  ],
  role='user'
), grounding_metadata=None, partial=None, turn_complete=None, finish_reason=None, error_code=None, error_message=None, interrupted=None, custom_metadata=None, usage_metadata=None, live_session
  parts=[
    Part(
```

Source Code verfügbar unter:
https://github.com/JensKohl/Lecture_Materials_Technical_Applications_Data_Management/blob/main/Notebooks/V12_Research_Agent.ipynb

Case Study
Agentic/ LLM-based application at scale

Case Study Agentic AI at scale

Übersicht Agentic AI stack



Quelle: Huyen: „AI Engineering", 2024, S. 38

Case Study Agentic AI at scale

Übersicht Anforderungen aus Sicht Kunde und Betreiber (Auszug)

Sicht **Kunde**

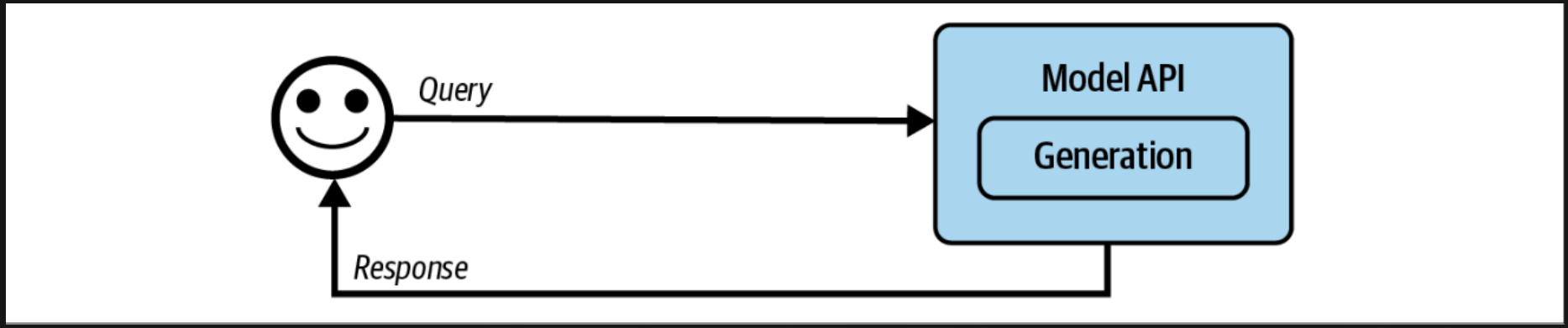
- Qualität Antworten: hilfreich, präzise, fehlerfreie Ergebnisse ohne Halluzinieren.
- Latenz: schnelle Rückmeldungen, auch bei komplexen Anforderungen
- Personalisierung: Agent kennt Präferenzen und berücksichtigt vergangene Interaktionen
- Datenschutz: sicherer Umgang mit persönlichen Daten des Anwenders
- Erklärbarkeit/ Transparenz: Nachvollziehbarkeit, warum Aktion x ausgeführt wurde.
- Zuverlässigkeit: Aufträge werden umgesetzt
- Korrigieren: Handlungen rückgängig machbar?

Sicht **Betreiber**

- Skalierbarkeit: stabiler Betrieb bei massiver, gleichzeitiger Nutzung
- Kosteneffizienz: aktives Begrenzen Token-Kosten (bspw. Vermeiden Endlosschleifen)
- Safety: Schutz vor Missbrauch interner/ externer Systeme durch Agenten und seine Handlungen
- Compliance: Revisionssichere Dokumentation und Einhalten rechtlicher Rahmenbedingungen.
- Security: nur Nutzung von User autorisierten Daten
- Observability: Überwachung Agent und Handlungen um Fehlerquellen zu identifizieren
- Wartbarkeit: Agent und Wissen ist aktualisierbar.
- Human in the Loop: Eskalationspfad für menschliche Freigaben bei kritischen Aktionen

Case Study Agentic AI at scale

Design einer skalierbaren Agentic AI-Anwendung: Schritt 1 – einfachste Architektur.

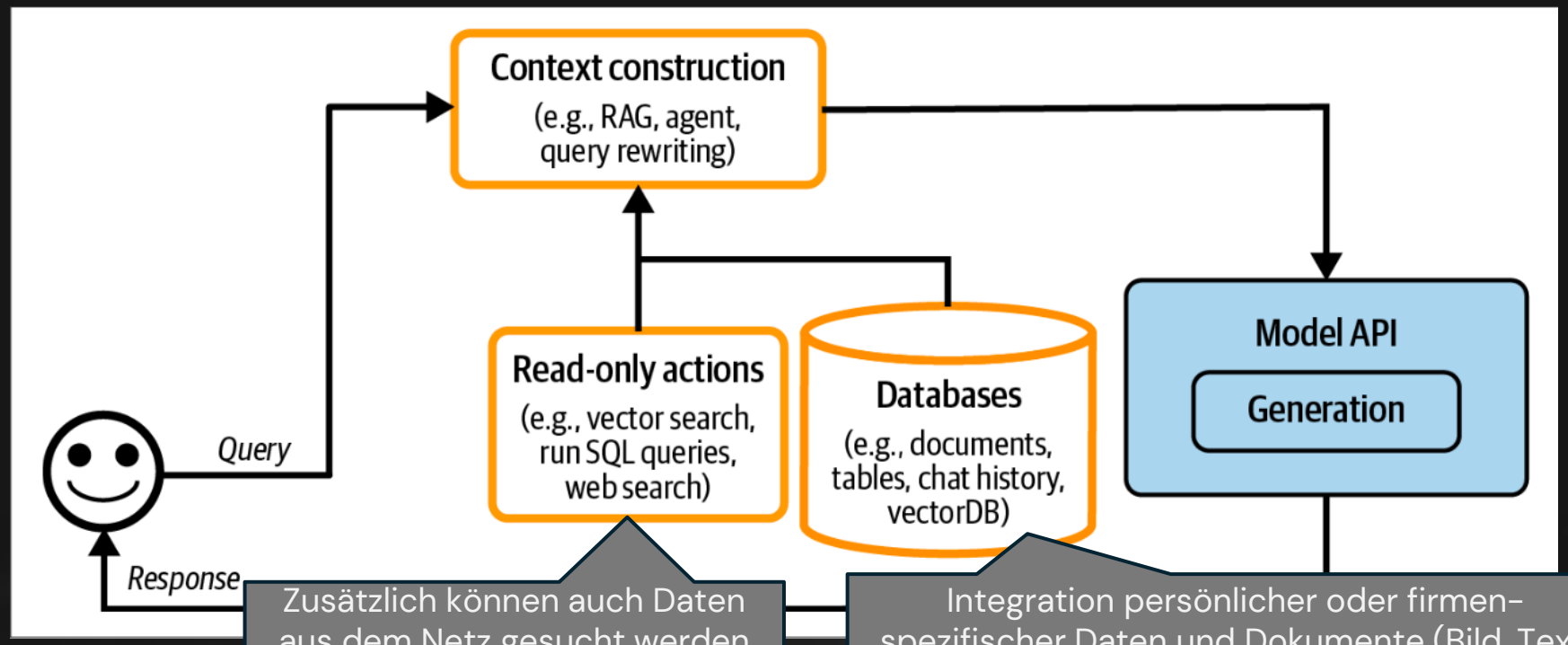


Einfachstes Design-Entwurf: Anfragen werden direkt an ein LLM geschickt, das nur Text generiert. Deckt aber kaum eine der Anforderungen an. Deshalb erweitern wir schrittweise.

Quelle: Huyen: „AI Engineering", 2024, Abb. 10-1

Case Study Agentic AI at scale

Design einer skalierbaren Agentic AI-Anwendung: Schritt 2 – Erweiterung Kontext.



Zusätzlich können auch Daten aus dem Netz gesucht werden

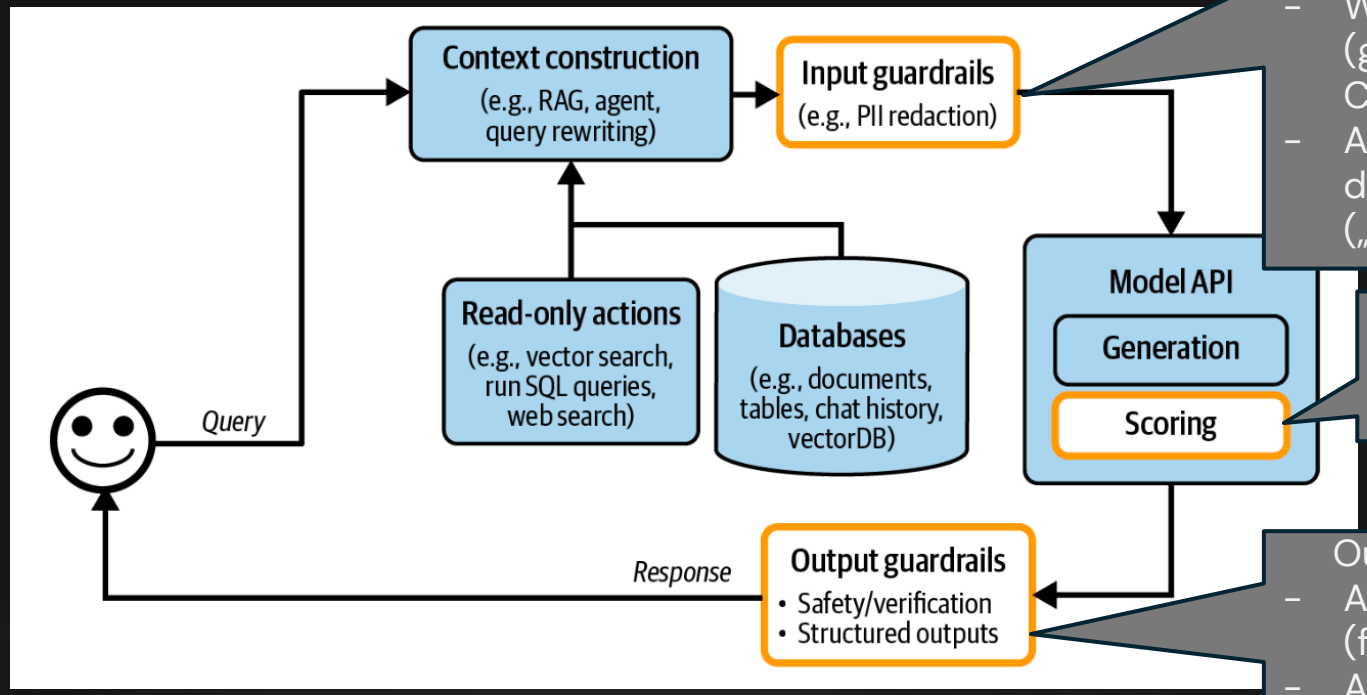
Integration persönlicher oder firmen-spezifischer Daten und Dokumente (Bild, Text, Audio, …) sowie Chat-Historie mit Agenten

Erweiterung Kontext ermöglicht qualitativ bessere Antworten, Personalisierung und Aktualisierung Wissen (ohne Training LLM).
Die Suche nach den Daten kann exakt (100% Gleichheit) oder semantisch (Ähnlichkeit) erfolgen.
Zugriff auf persönliche Daten erfordert höheren Aufwand für Sicherstellen Datenschutz, rechtliche Vorgaben und deren Dokumentation.

Quelle: Huyen: „AI Engineering", 2024, Abb. 10-2

Case Study Agentic AI at scale

Design einer skalierbaren Agentic AI-Anwendung: Schritt 3 – Guardrails



Input-Guardrails verhindern:
- Weitergeben privater Daten (ggf. Anonymisieren) und ähnl. Compliance-Verfehlungen
- Ausführen von Prompts, die dem System schaden können („Prompt Injection“)

Bewertet Ausgabe auf Fehlerhaftigkeit oder Kritikalität

Output-Guardrails verhindern:
- Ausgeben fehlerhafter Daten (falsches Format, Typ, Ergebnis)
- Ausgeben problematischer Daten (rassistisch, sexistisch, …)

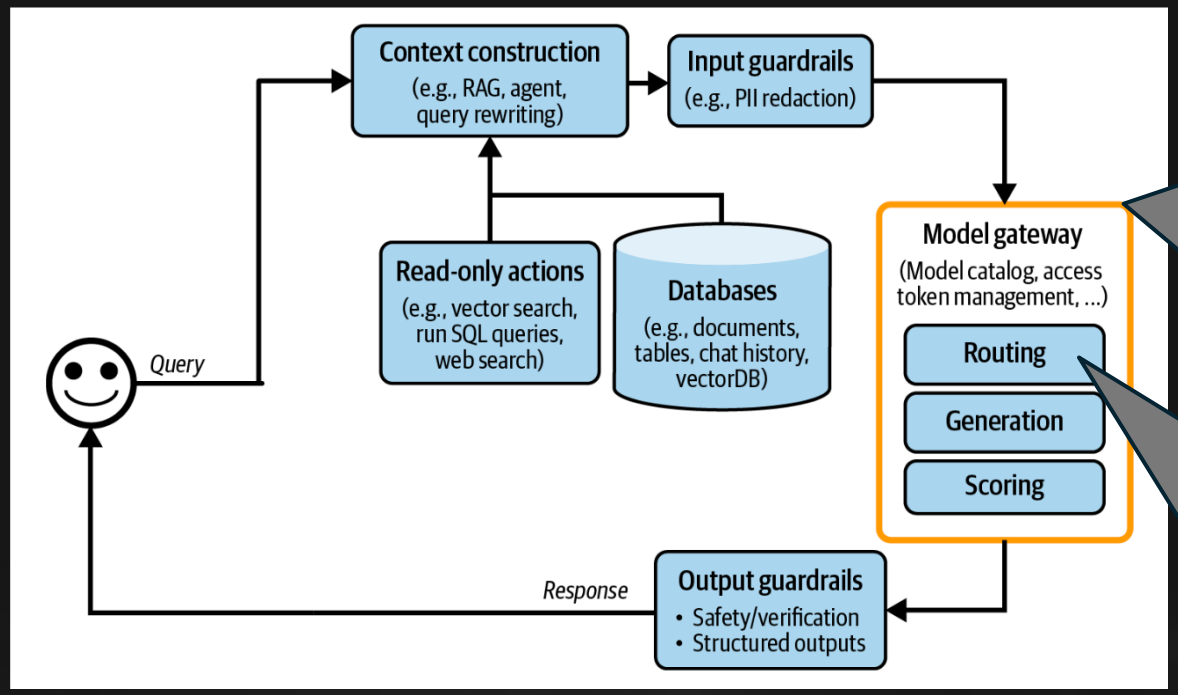
Zudem Einbindung Mensch bei kritischen Themen möglich

Guardrails ermöglichen Risikominderung, in dem Ein- und Ausgaben bewertet werden. Zudem kann Mensch eingebunden werden. Durch die Prüfung ist eine höhere Qualität/ Zuverlässigkeit möglich auf Kosten von höherer Latenz.

Quelle: Huyen: „AI Engineering“, 2024, Abb. 10-4

Case Study Agentic AI at scale

Design einer skalierbaren Agentic AI-Anwendung: Schritt 4 – Model routing und Gateway



Gateway erlaubt:
- Einsatz verschiedener Modelle
- einheitliche Schnittstelle zu verfügbaren Modellen
- Zentrale Zugriffsstelle und somit Zugriffskontrolle
- Kostenmanagement

Zuweisung an passendstes Modell erfolgt über „intent classifier“, d.h. ML-Modell, das vorhersagt, was Kunde machen will.
Bei Unsicherheit kann Classifier bei User nachfragen.
Anfrage kann auch abgelehnt werden, falls problematisch.
Routing modell oft kleineres LLM-Modell

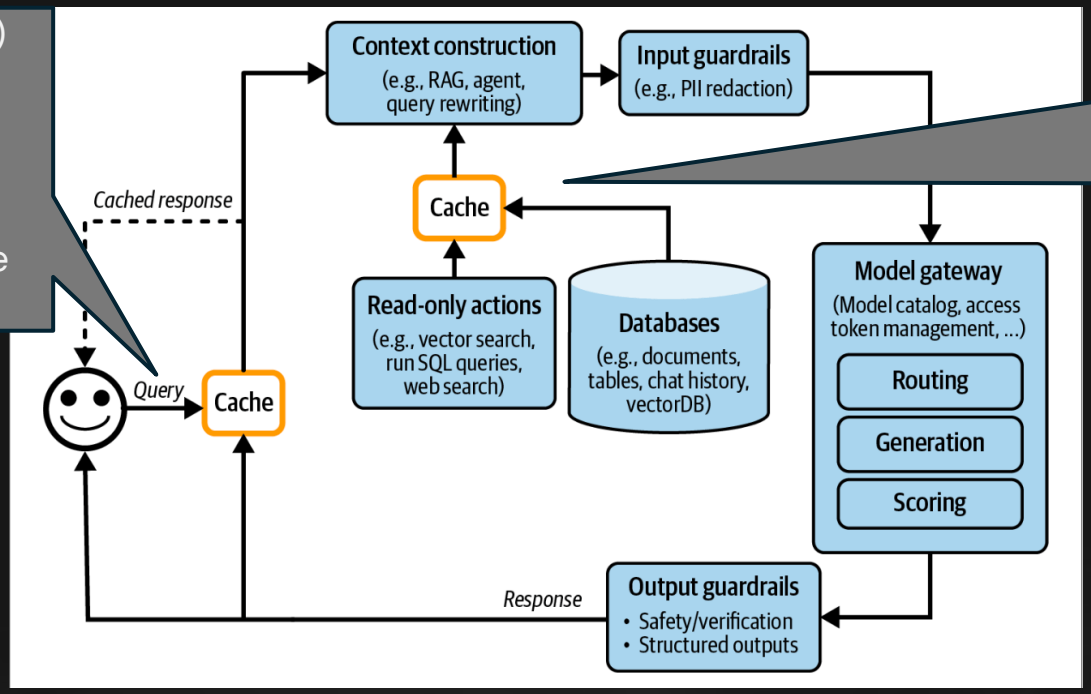
Gateway erlaubt einheitlichen Zugriff auf verschiedene Modelle und erleichtert so Wartbarkeit, Erweiterbarkeit und Sicherheit.
Router ermöglicht die Auswahl spezifischer Modelle für höhere Qualität oder kleinerer Modelle um Kosten/ Latenz zu reduzieren.

Quelle: Huyen: „AI Engineering“, 2024, Abb. 10-7

Case Study Agentic AI at scale

Design einer skalierbaren Agentic AI-Anwendung: Schritt 5 – Cache

Prüft, ob exakt gleiche („exact caching“) oder ähnliche („semantic caching“) Anfrage bereits gestellt wurde.
Falls ja, dann Rückgabe Ergebnis ohne erneute Anfrage an das Modell.
Falls nein, dann Ausführung der Anfrage und Cachen des Ergebnis



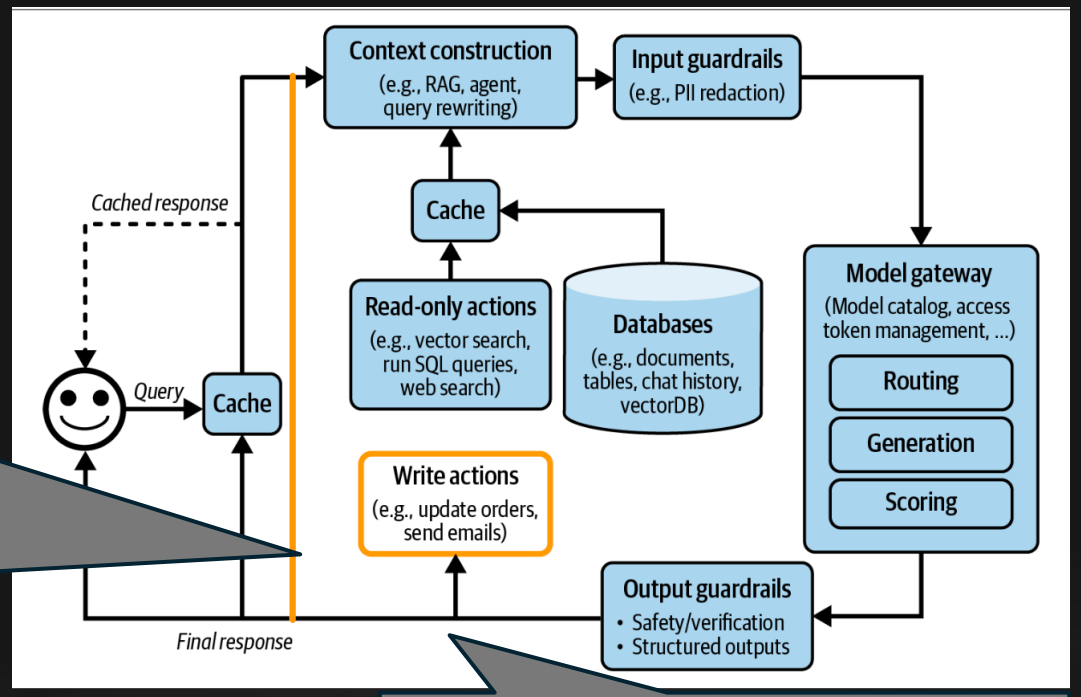
Bei Anfragen an das Modell wird der Key und Value für die Anfrage in einer Datenbank gespeichert.
Dies ermöglicht die Reduktion von Kosten und Erhöhung Geschwindigkeit, da diese Werte nicht neu berechnet werden müssen

Caches reduzieren die Latenz und die Kosten, erhöhen aber den Ressourcenbedarf und bieten potentielle Security-Angriffspunkte.
Durch Manipulation oder unbefugtes Auslesen Caches sowie Umgehen Zugriffsrechten kann Integrität und Vertraulichkeit System gefährden.
Exact Caching benötigt weniger Ressourcen, Semantic Caching maximiert Trefferrate durch Erkennen ähnlicher Anfragen.

Quelle: Huyen: „AI Engineering“, 2024, Abb. 10-8

Case Study Agentic AI at scale

Design einer skalierbaren Agentic AI-Anwendung: Schritt 6 – Agentische Fähigkeiten



ReAct-Zyklus Dauerschleife, bis vom User gestellte Aufgabe erfüllt:

- **Reason:** Agent reflektiert aktuellen Status und plant nächsten logischen Schritt.
- **Act:** Agent wählt passendes Tool und führt die Aktion aktiv aus.
- **Observe:** Auswerten Ergebnis für Entscheidung: Finale Antwort geben oder erneut zu Reason?

Ausführen von Aktionen. Dadurch können permanente Änderungen erreicht werden.

Durch Tools und das Rückführen von Ergebnissen können Agenten komplexere Pläne eigenständig ausführen, sich selbst korrigieren und auch reale Interaktionen mit der Umgebung führen.

Quelle: basierend auf Huyen: „AI Engineering", 2024, Abb. 10-10

Case Study Agentic AI at scale

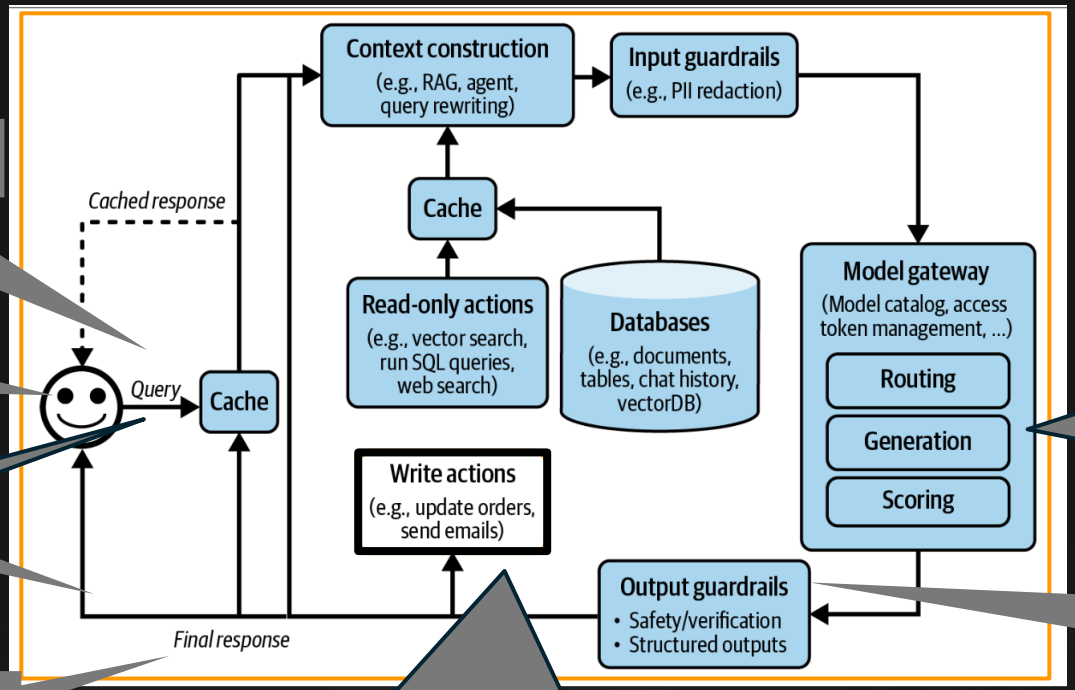
Design einer skalierbaren Agentic AI-Anwendung: Schritt 7 – Monitoring & Observability

Metrik: Länge User Anfragen?

Metrik: wie oft brach User Anfrage ab? Wie oft gab User Feedback daß unzufrieden?

Loggen User Anfragen und finalen Antwort

Metrik: Länge der finalen Ausgabe



Loggen der Aktionen (welches Tool mit welchem Inhalt ausgeführt?

Loggen, des Routing (welches Modell gewählt), was Modell generiert (z.B. Reasoning), Scoring

Metrik: wie oft falscher Output (Format/ Inhalt)? Wie oft Guardrails verletzt?

Observability bietet Identifikation Optimierungspotentiale und Risikominderung durch (frühzeitiges) Erkennen anbahnender Probleme. Während Metriken Informationen aggregieren, erlauben Logs genaue Ursachenanalyse – weshalb gerade anfänglich umfassende Logs wichtig sind. Die Kombination aus beidem erlaubt Transparenz, lückenlose Überwachung des Systems, frühzeitiges Steuern von Abweichungen.

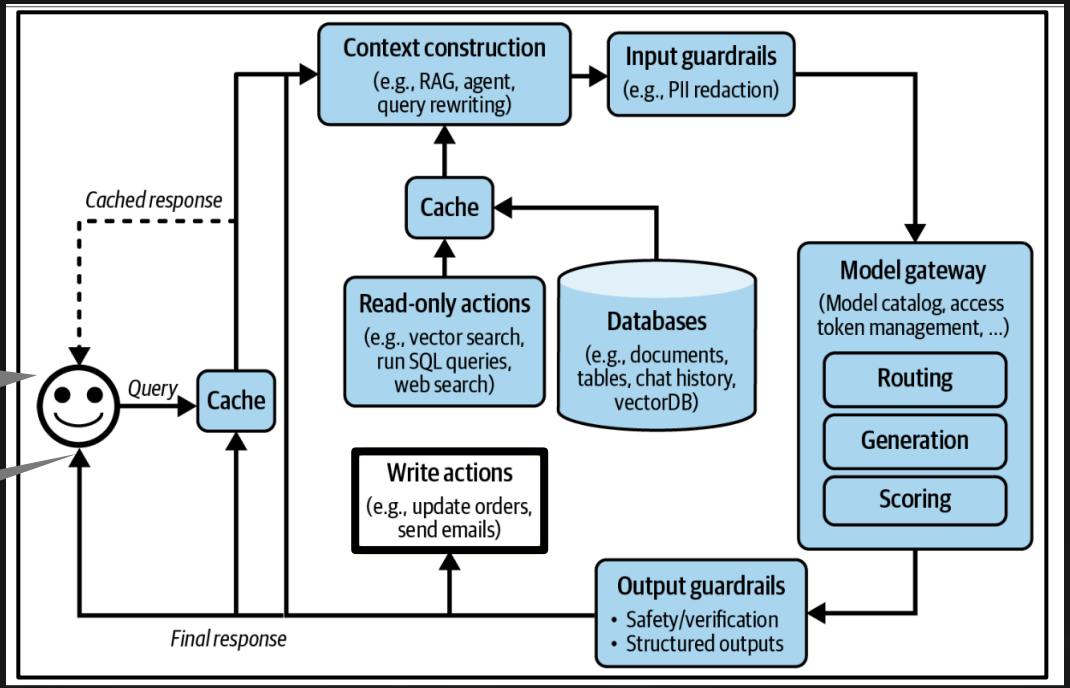
Quelle: basierend auf Huyen: „AI Engineering", 2024, Abb. 10-10

Case Study Agentic AI at scale

Design einer skalierbaren Agentic AI-Anwendung: Schritt 8 – Analysiere User Feedback

Explizites Feedback: User Möglichkeit anbieten, direkt Feedback zu geben (bspw. Daumen hoch), Auswerten schriftliches Feedback, Beschwerden

Implizites Feedback: aus Aktionen des Kunden Feedback schließen (bspw. plötzlicher Abbruch Gespräch, Wahl anderes Modelles, Erneutes Eingeben gleichen/ ähnlichen Prompts, …)



Sammeln und Auswerten von implizitem & explizitem Kundenfeedback ermöglicht kont. Verbesserung Output-Qualität und Personalisierung Das Bestimmen von Kriterien für implizites Feedback ist anspruchsvoller, ermöglicht jedoch automatisierten Verbesserungsprozess (**Data Flywheel**).

Quelle: basierend auf Huyen: „AI Engineering“, 2024, Abb. 10-10

Literatur und weitere Quellen

Veröffentlichungen:

- Z. Xi et al.: “The rise and potential of large language model based agents: a survey,” Science China Information Sciences, vol. 68, no. 2, p.121101, 2025.
- Wei et al.:”Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”, 2022.
- Yao et al.: “ReAct: Synergizing Reasoning and Acting in Language Models”, 2023
- Google: “Introduction to agents”, 2025. Link
- Weng: “LLM Powered Autonomous Agents“, 2023. Link
- Chip Huyen: “AI Engineering“, 2024.

Frameworks:

- Anthropic Model Context Protocol (Quasi Standard für Verknüpfung Agenten mit externen Systemen): Link
- Google Agent to Agent Protokoll. Link
- LangGraph: Framework für Bau von Agenten. Link
- Generative AI Toolkit: Framework für Bau und Testen von Agenten. Link
- CrewAI: Framework für den Bau von Multi-Agenten. Link

Backup